# COSC2123/3119: Algorithms & Analysis

## Course Overview

Elham Naghizade

RMIT University
Email : e.naghizade@rmit.edu.au

Lecture 1

# Outline


1. Preliminaries
2. What is an algorithm and motivation for its study
3. Abstract Data Types
4. Data Structures
5. Summary

# Overview


1. Preliminaries
2. What is an algorithm and motivation for its study
3. Abstract Data Types
4. Data Structures
5. Summary

# Staff

**Course coordinator & Lecturer:** Elham Naghizade
**Email:** e.naghizade@rmit.edu.au



**Team of workshop demonstrators:** Ahmed, Ed (head tutor), Imesh, Kashif, Nestor, Ying & Yueqing

**Course Homepage:** Go to Canvas,
`https://rmit.instructure.com/courses/141229`
(COSC2123/3119, 2025 Semester 1).

# General Course Goals

## Key Goal

Study algorithms and their analysis in order to **find practical solutions to real-world problems**

# General Course Goals

## Key Goal

Study algorithms and their analysis in order to **find practical solutions to real-world problems**

Our goals are:

- **Theory:**
  - Learn a variety of important algorithms
  - Understand how to determine algorithm complexity, that is, the estimate running time

# General Course Goals

## Key Goal

Study algorithms and their analysis in order to **find practical solutions to real-world problems**

Our goals are:

- **Theory:**
  - Learn a variety of important algorithms
  - Understand how to determine algorithm complexity, that is, the estimate running time
- **Practice:**
  - Learn to *implement* various algorithms and data structures to solve a variety of problems
  - Understand how to measure the *actual* running time of an algorithm *empirically*

# Why study Algorithms, Data Structures and their evaluation?

In order to solve problems, including building computer programs to do so, we need to have (algorithmic) tools and evaluate how efficient the tools are.

Why do we need an algorithmic toolset and learn how to evaluate their efficiency?



# Why study Algorithms, Data Structures and their evaluation?

In order to solve problems, including building computer programs to do so, we need to have (algorithmic) tools and evaluate how efficient the tools are.



Why do we need an algorithmic toolset and learn how to evaluate their efficiency?

**Answer:**

- Need to know how the algorithms work and how to measure their efficiency in order to select the right one for the right problem/job.

# Why study Algorithms, Data Structures and their evaluation?

In order to solve problems, including building computer programs to do so, we need to have (algorithmic) tools and evaluate how efficient the tools are.



Why do we need an algorithmic toolset and learn how to evaluate their efficiency?

**Answer:**

- Need to know how the algorithms work and how to measure their efficiency in order to select the right one for the right problem/job.
- Designing new algorithms or modify exiting ones to solve your problems.

# Why study Algorithms, Data Structures and their evaluation?

In order to solve problems, including building computer programs to do so, we need to have (algorithmic) tools and evaluate how efficient the tools are.



Why do we need an algorithmic toolset and learn how to evaluate their efficiency?

**Answer:**

- Need to know how the algorithms work and how to measure their efficiency in order to select the right one for the right problem/job.
- Designing new algorithms or modify exiting ones to solve your problems.
- **It makes you a better problem solver and programmer!**

# Prequisites

- This course is **not** about programming, but we do need a language for the practical parts:
  - Python: At a minimum, you should be comfortable with *basic data types and structures, defining new classes and data types*.

# Prequisites

- This course is **not** about programming, but we do need a language for the practical parts:
  - Python: At a minimum, you should be comfortable with *basic data types and structures, defining new classes and data types*.
  - Warning: The assignment and labs are all in Python

# Prequisites

- This course is **not** about programming, but we do need a language for the practical parts:
  - Python: At a minimum, you should be comfortable with *basic data types and structures, defining new classes and data types*.
  - Warning: The assignment and labs are all in Python
- Prequisites:
  - *Programming Fundamentals* or *Further Programming*, or *Programming Bootcamp 2 Advanced Programming Techniques*, which teach Python and concepts helpful for this course.

# Maths :)

This course has some **maths** in it.

# Maths :)

This course has some **maths** in it.

We need to count how many times certain operations occur to analysis algorithms.

- Equations (summations, recursions). $\sum_{i=0}^{n} 2$.
- Arithmetic and partial sum simplifications. $\sum_{i=0}^{n} 2 = 2n + 2$.

If you are uncomfortable with above, it is advisable that you review your maths courses.

# Maths :)

This course has some **maths** in it.

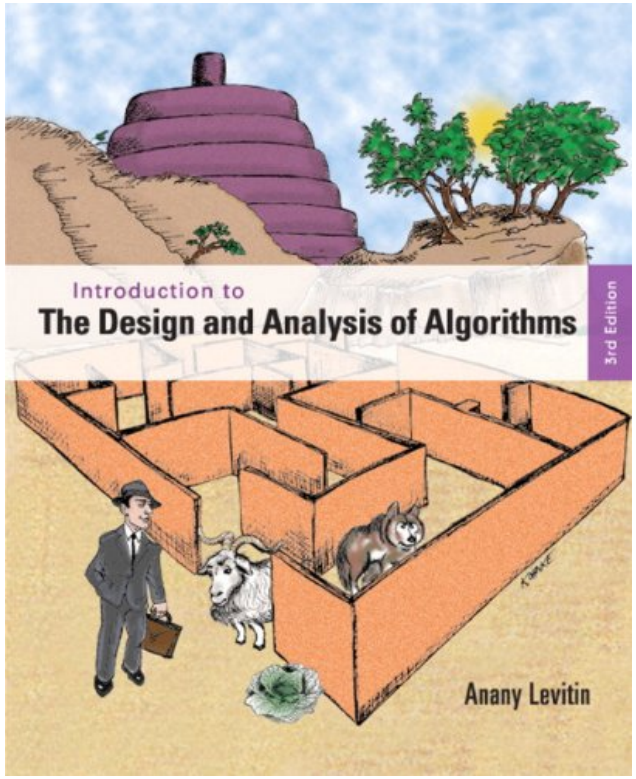
We need to count how many times certain operations occur to analysis algorithms.

- Equations (summations, recursions). $\sum_{i=0}^{n} 2$.
- Arithmetic and partial sum simplifications. $\sum_{i=0}^{n} 2 = 2n + 2$.

If you are uncomfortable with above, it is advisable that you review your maths courses.



# Official Textbook




A. Levitin. *Introduction to the design and analysis of algorithms*. Addison Wesley, Boston, third edition, 2012, ISBN-13: 9780132316811.

# Other Reference Books




T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, third edition, 2008.

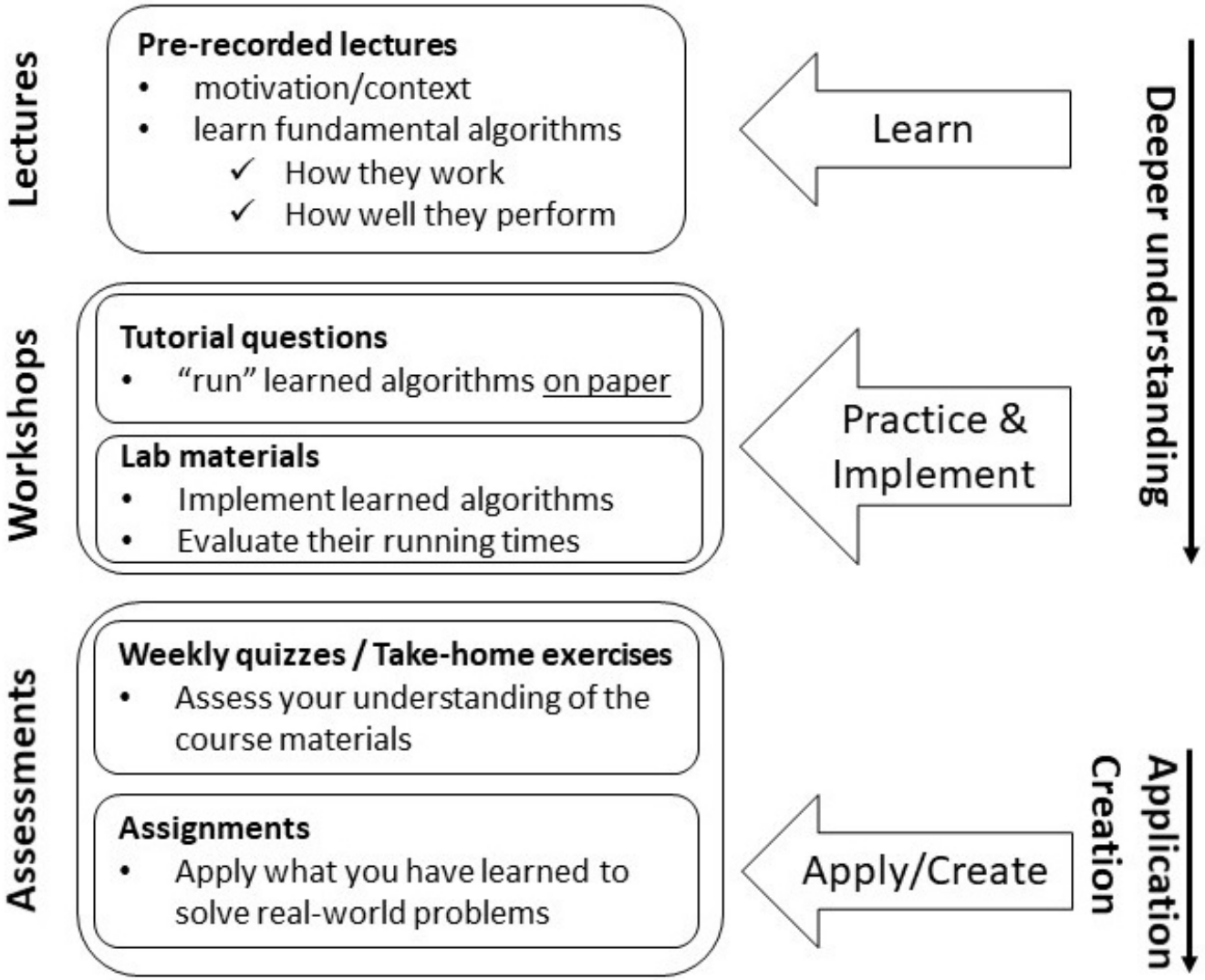


J. Kleinberg and É. Tardos. *Algorithm Design*. Pearson, Boston, 2006.

# Course Syllabus

| Lecture | Topic | Reading |
|---|---|---|
| 1 | Introduction | Levitin – Ch 1 |
| 2 | Basic Data Structures & Algorithmic Analysis | Levitin – Ch 2 |
| 3 | Algorithmic Analysis | Levitin – Ch 2 |
| 4 | Brute Force | Levitin – Ch 3 |
| 5 | Decrease & Conquer | Levitin – Ch 4 |
| 6 | *In-person Mid-semseter Challenge* | |
| 7 | Divide & Conquer | Levitin – Ch 5 |
| 8 | Transform & Conquer | Levitin – Ch 6 |
| 9 | Dynamic Programming | Levitin – Ch 8 |
| 10 | Greedy Techniques | Levitin – Ch 9 |
| 11 | Incremental Improvement | Levitin – Ch 10 |
| 12 | Time & Space Tradeoffs | Levitin – Ch 7 |

# Course Structure



# Assessment

10% of your assessment will come from weekly online quizzes that you can attempt on Canvas.

20% of your assessment will come from the mid-semester challenge.

30% of your assessment will come from an assignment.

40% of your assessment will come from the End-of-semester timed and timetabled exercises.

# Weekly Quizzes

- Every week, there will be a 5 question quiz.

# Weekly Quizzes

- Every week, there will be a 5 question quiz.
- A total of 12 quizzes, one per week, will be available for you to complete through Canvas.

# Weekly Quizzes

- Every week, there will be a 5 question quiz.
- A total of 12 quizzes, one per week, will be available for you to complete through Canvas.
- Each quiz contributes up to 1% of your final course mark. If you complete more than 10 quizzes, we take your 10 best results to compute the quiz mark component

# Weekly Quizzes

- Every week, there will be a 5 question quiz.
- A total of 12 quizzes, one per week, will be available for you to complete through Canvas.
- Each quiz contributes up to 1% of your final course mark. If you complete more than 10 quizzes, we take your 10 best results to compute the quiz mark component .
- For each quiz, if you score at least 4 out of 5, you will get will get the 1% mark, if you score 3 out of 5, you will get 0.5% marks, and if you score less than 3, you will get 0 marks for that quiz.

# Weekly Quizzes

- Every week, there will be a 5 question quiz.
- A total of 12 quizzes, one per week, will be available for you to complete through Canvas.
- Each quiz contributes up to 1% of your final course mark. If you complete more than 10 quizzes, we take your 10 best results to compute the quiz mark component .
- For each quiz, if you score at least 4 out of 5, you will get will get the 1% mark, if you score 3 out of 5, you will get 0.5% marks, and if you score less than 3, you will get 0 marks for that quiz.
- Each quiz must be completed before 11:59 PM on the Sunday of that week.

# Change I: Mid-semester Challenge

- It will be a pen & paper test, held in person with no alternative format available.
- Scheduled during Week 6 lectorial sessions.
- You must attend the session in which you are enrolled.

# Change II: We use multiple platforms!

- Go to *Canvas* for course material - lecture recordings, lecture, lectorial and workshop notes - as well as quizzes. You can also find general information about the course and university resources.

# Change II: We use multiple platforms!

- Go to *Canvas* for course material - lecture recordings, lecture, lectorial and workshop notes - as well as quizzes. You can also find general information about the course and university resources.
- Go to *the Ed Discussion Forum* to read the FAQ pages and post your questions.

# Change II: We use multiple platforms!

- Go to *Canvas* for course material - lecture recordings, lecture, lectorial and workshop notes - as well as quizzes. You can also find general information about the course and university resources.
- Go to *the Ed Discussion Forum* to read the FAQ pages and post your questions.
- Go to *GitHub* to access and submit your assignment.

# Sources of help (We are here to help!)

- *Contact the lecturer* if you are having difficulties which may prevent you from completing the course as soon as possible.

# Sources of help (We are here to help!)

- *Contact the lecturer* if you are having difficulties which may prevent you from completing the course as soon as possible.
- *Post your questions to the Ed Discussion Forum* to get answers (within 24 hours) in weekdays.

# Sources of help (We are here to help!)

- *Contact the lecturer* if you are having difficulties which may prevent you from completing the course as soon as possible.
- *Post your questions to the Ed Discussion Forum* to get answers (within 24 hours) in weekdays.
- Ask tutors and lab demonstrators *during the workshops*!

# Sources of help (We are here to help!)

- *Contact the lecturer* if you are having difficulties which may prevent you from completing the course as soon as possible.
- *Post your questions to the Ed Discussion Forum* to get answers (within 24 hours) in weekdays.
- Ask tutors and lab demonstrators *during the workshops*!
- 2 weekly consultations times per week, one with myself and one with Ed (head tutor), see `https://rmit.instructure.com/courses/141229/pages/teaching-team?module_item_id=7131311` for times.

# Sources of help (We are here to help!)

- *Contact the lecturer* if you are having difficulties which may prevent you from completing the course as soon as possible.
- *Post your questions to the Ed Discussion Forum* to get answers (within 24 hours) in weekdays.
- Ask tutors and lab demonstrators *during the workshops*!
- 2 weekly consultations times per week, one with myself and one with Ed (head tutor), see `https://rmit.instructure.com/courses/141229/pages/teaching-team?module_item_id=7131311` for times.
- You can also help each other!

# How to succeed in the course?

*"I heard the course is really hard."*

# How to succeed in the course?

*"I heard the course is really hard."* This course can be different to what you done before, but it is not as difficult as what you may have heard.

# How to succeed in the course?

*"I heard the course is really hard."* This course can be different to what you done before, but it is not as difficult as what you may have heard.

**Advice:**

- From previous semesters, those that studied throughout the semester, did **well**.
- Those that attended and engaged in tutorials and labs, did **well**.
- Those that crammed for exams, not **so well**.
- Do *something* for the course everyday: read, code, work a problem . . . this is **not** a course you can **cram**.

# Extensions/Special Considerations

- Where possible, let us know if you may be delayed in submitting an assignment or other assessable material. Please do **not** wait until the day before it is due.
- Extensions are only granted in exceptional circumstances and require the *official process of "Special Consideration"* to be followed.

# Week 1 Overview

## Levitin – The design and analysis of algorithms

This week we will be covering the material from Chapter 1.

Learning outcomes:

- Understand the concept of algorithms and data structures and the motivation behind their analysis.
- Learn about different abstract data types and data structures (expanded upon in subsequent classes).

# Overview


1. Preliminaries
2. What is an algorithm and motivation for its study
3. Abstract Data Types
4. Data Structures
5. Summary

# What is an algorithm?

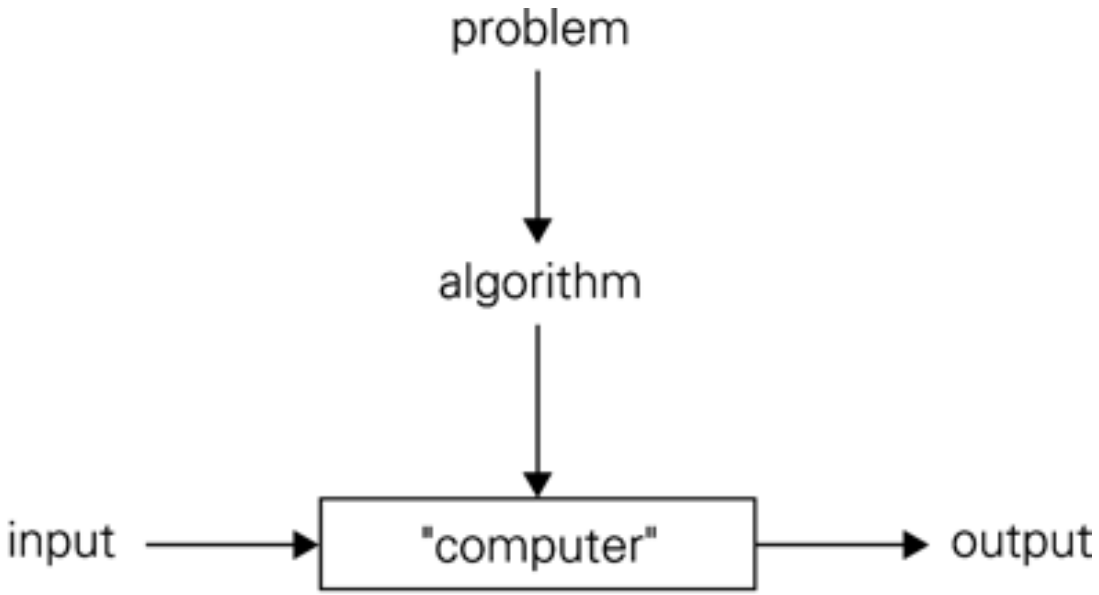



**FIGURE 1.1** Notion of algorithm

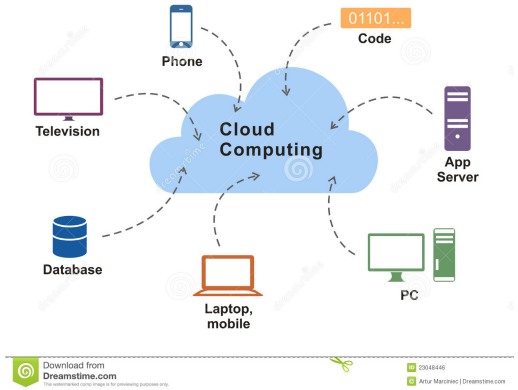
An algorithm is a sequence of (unambiguous) instructions/steps for solving a problem.

# Examples of Algorithms ... ?

















# Example: Euclid's Algorithm

Problem : Find $\textbf{GCD}(m, n)$, the greatest common divisor of two non-negative integers $m$ and $n$.

Examples: $\textbf{GCD}(60, 24) = 12$; $\textbf{GCD}(60, 0) = 60$; $\textbf{GCD}(0, 0) = ?$

# Example: Euclid's Algorithm

Problem : Find $\mathbf{GCD}(m, n)$, the greatest common divisor of two non-negative integers $m$ and $n$.

Examples: $\mathbf{GCD}(60, 24) = 12$; $\mathbf{GCD}(60, 0) = 60$; $\mathbf{GCD}(0, 0) = ?$

## Solution (*sketch*)

Euclid's algorithm is based on repeated application of the equality $\mathbf{GCD}(m, n) = \mathbf{GCD}(n, m \bmod n)$ until the second number ($n$) becomes 0. The answer is then the first number ($m$).

Example : $\mathbf{GCD}(60, 24) = \mathbf{GCD}(24, 12) = \mathbf{GCD}(12, 0) = 12$

# Euclid's Algorithm Pseudocode

ALGORITHM **Euclid** $(m, n)$
// Computes **GCD**$(m, n)$ by Euclid's algorithm
// INPUT : Two non-negative integers $m$ and $n$
// OUTPUT : Greatest Common Divisor of $m$ and $n$

1: **if** $n > 0$ **then**
2: return **GCD**$(n, m \mod n)$
3: **end if**
4: **return** $m$

# Euclid's Algorithm

Alternative approaches to this problem?

Is it more "efficient"?

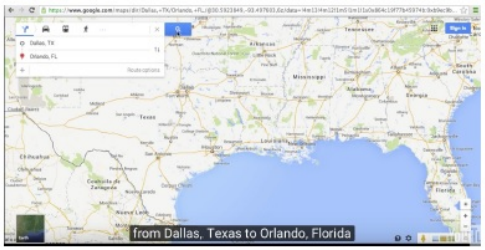
# GCD Alternative (Sketch)

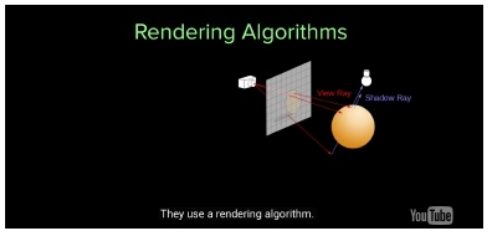
**Alternative - Common primes:**

**Step 1** Find the prime factors of $m$.
**Step 2** Find the prime factors of $n$.
**Step 3** Identify all the common factors in the two prime expansions from Step 1 and Step 2.
**Step 4** Compute the product of all common factors and return it as the greatest common divisor.

Example:
$60 = 2 \cdot 2 \cdot 3 \cdot 5$
$24 = 2 \cdot 2 \cdot 2 \cdot 3$
$\textbf{GCD}(60, 24) = 2 \cdot 2 \cdot 3 = 12$

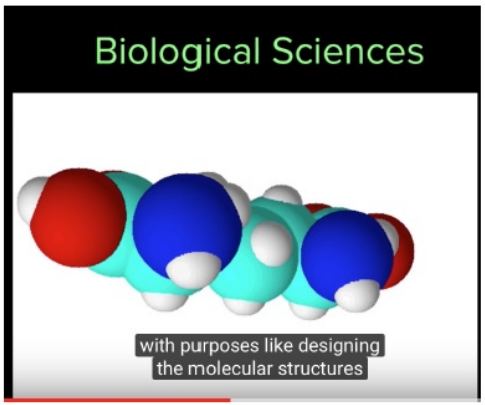
# Motivation for studying algorithms



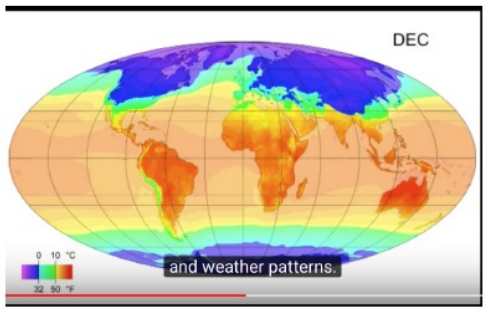













▸ Video

# Overview


1. Preliminaries
2. What is an algorithm and motivation for its study
3. Abstract Data Types
4. Data Structures
5. Summary

# Abstract Data Types and Data structures

Algorithms process input data, produce and process intermediate data, then output some results, which could be stored as data.

# Abstract Data Types and Data structures

Algorithms process input data, produce and process intermediate data, then output some results, which could be stored as data.

But we do not want to consider algorithms for each type of input data.

Instead we seek to talk about the input data, the intermediate data, and the output data in terms of general characteristics and operations the data should possess.

These data abstractions are called Abstract data types (ADT).

# Abstract Data Types and Data structures

Algorithms process input data, produce and process intermediate data, then output some results, which could be stored as data.

But we do not want to consider algorithms for each type of input data.

Instead we seek to talk about the input data, the intermediate data, and the output data in terms of general characteristics and operations the data should possess.

These data abstractions are called Abstract data types (ADT).

- An ADT can be defined by the collection of common operations that are accessed through an *interface* and its characteristics.

# Abstract Data Type

Data structure is the implementation of an abstract data type.

# Abstract Data Type

Data structure is the implementation of an abstract data type.

Example:

- Integer (-4,-1, 0, 29) is an abstract data type. Its operations include add, subtract, multiply etc.
- Bit vectors is a data structure. It is an implementation of an integer.
- Hexadecimal vectors is a data structure. It is an implementation of an integer.

# Abstract Data Type

Data structure is the implementation of an abstract data type.

Example:

- Integer (-4,-1, 0, 29) is an abstract data type. Its operations include add, subtract, multiply etc.
- Bit vectors is a data structure. It is an implementation of an integer.
- Hexadecimal vectors is a data structure. It is an implementation of an integer.

Motivation:

- We can specify the input and data forms of our algorithms in terms of ADT, and not worry about the actual implementation until we need to implement it.
- We can change data structure implementations to cater for the problem at hand.

# Fundamental ADTs

- set
- sequences
- dictionary/map
- stack
- queue
- priority queue
- graph
- tree

# Sets

## Sets

A *set* is a collection of distinguishable objects, often called *members* or *elements*.

- Sets can be finite or infinite, and do not impose any ordering on the members.
- Typical operations: add, remove, search
- Each element may only appear in the set once. If elements may appear multiple times, the resulting collection is referred to as a bag or multiset.

# Sets

## Example (Sets)

Simple examples of sets include:

1. binary : $S_1 = \{0, 1\}$,
2. character : $S_2 = \{c, a, y, s, t\}$,
3. word : $S_3 = \{$ car, house, man, sat, truck $\}$,
4. delimiter : $S_4 = \{\{!\}, \{;\}, \{?\}, \{.\}\}$,
5. variable-length, prefix-free bit sequences: $S_5 = \{0, 10, 110, 1110\}$.

# Sequences

## Sequences

A *sequence* is a collection of elements in which the order of the elements must be maintained, and elements can occur any number of times.

- A sequence is simply a multiset in which the order is important.
- Typical operations: add, remove, search
- In computer science, can also be referred to as a list (ordered collection of elements).

# Sequences

## Example (Sequences)

Examples of sequences include:

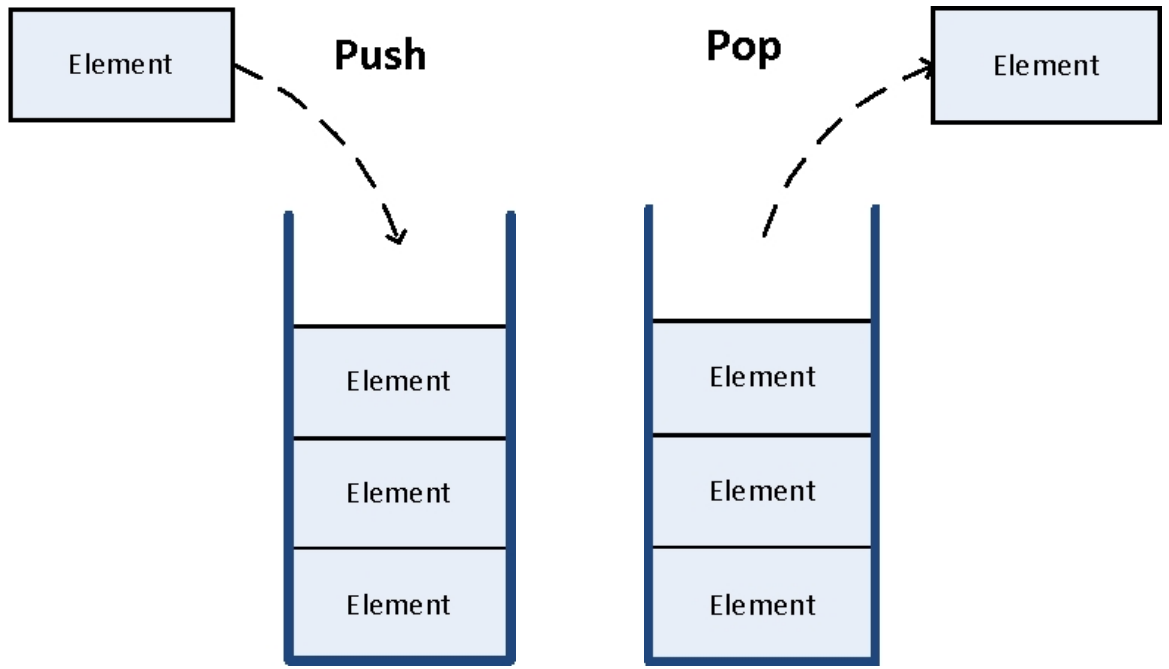
1. Binary : $\mathcal{T}_1 =$ “101010110001”,
2. English : $\mathcal{T}_2 =$ “The red car belongs to me.”,
3. Genomic : $\mathcal{T}_3 =$ “gattcaggaatccgccggtaacgcgcatataattt”.

# Stack

## Stack

A *stack* is a collection with two defining operations, *push* and *pop*. Push adds new element at top of stack, pop removes element at top from stack.

Stack ADT implements LIFO principle (last in, first out).



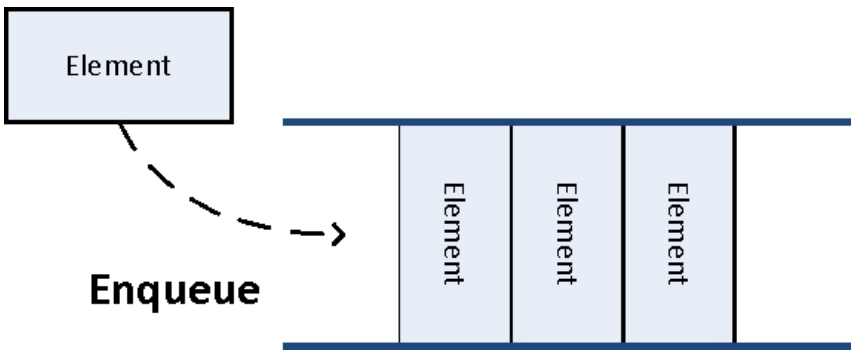

(a) push (b) pop

# Queue

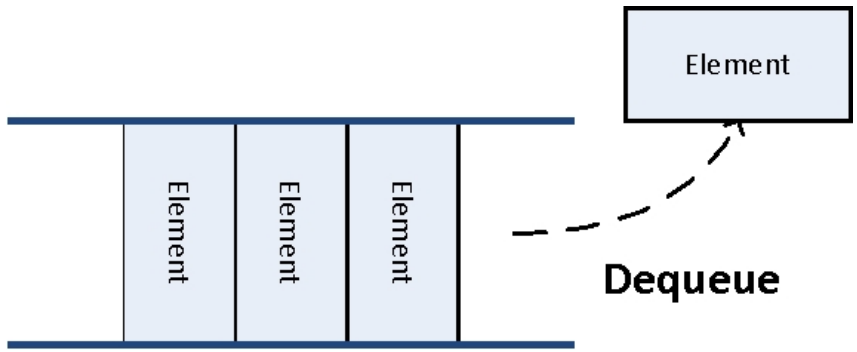
## Queue

A *queue* is a collection with two defining operations, *enqueue* and *dequeue*. Enqueue adds new element at back of queue, dequeue removes element at front from queue.

Queue ADT implements FIFO principle (first in, first out).




(c) enqueue




(d) dequeue

# Priority Queue

## Queue

A priority queue is a queue that has a priority associated with each element it holds. Enqueue is the same as a (non-priority) queue, but for dequeue, the element with the highest priority is removed first.

# Priority Queue

## Queue

A priority queue is a queue that has a priority associated with each element it holds. Enqueue is the same as a (non-priority) queue, but for dequeue, the element with the highest priority is removed first.

Example: Queue for boarding planes, passengers who are members of the airlines or elderly have priority board plane (dequeue).

# Dictionaries/Maps

## Dictionary/Maps

A *dictionary/map* is a collection of *(key,value)* pairs, such that each key can only appear once in the dictionary/map.

# Dictionaries/Maps

## Dictionary/Maps

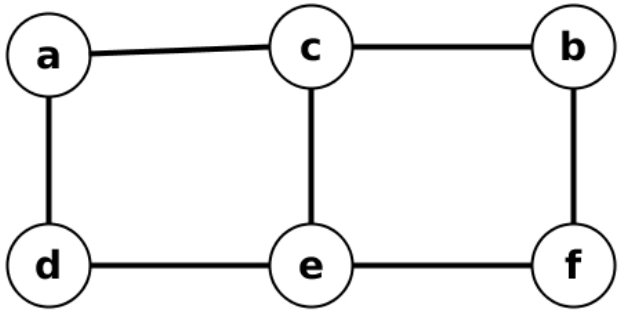
A *dictionary/map* is a collection of *(key,value)* pairs, such that each key can only appear once in the dictionary/map.

## Example (Dictionary/map)

Example of a dictionary of current leaders and countries they led:

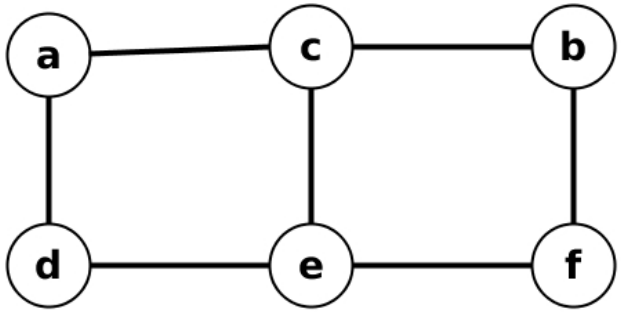
| **key** | **value** |
|---|---|
| Scott Morrison | Australia |
| Xi Jinping | China |
| Boris Johnson | UK |
| Joe Biden | US |

# Graphs




A graph $G = \langle V, E \rangle$ is defined by a pair of two sets: a finite set $V$ of items called vertices and a set $E$ called edges, representing links/relations/connections between pairs of vertices.

# Graphs




A graph $G = \langle V, E \rangle$ is defined by a pair of two sets: a finite set $V$ of items called vertices and a set $E$ called edges, representing links/relations/connections between pairs of vertices.

Example:
$V = \{a, b, c, d, e, f\}$
$E = \{(a, d), (a, c), (d, e), (c, e), (c, b), (e, f), (b, f)\}$

# Graphs

- A graph $G$ is undirected if the edges do not have a direction (i.e., all of the pairs of vertices in $E$ are unordered).
- A graph $G$ is directed if the edges from a direction (i.e., all of the pairs of vertices in $E$ have an ordering imposed).

# Graphs

- A graph $G$ is undirected if the edges do not have a direction (i.e., all of the pairs of vertices in $E$ are unordered).
- A graph $G$ is directed if the edges from a direction (i.e., all of the pairs of vertices in $E$ have an ordering imposed).

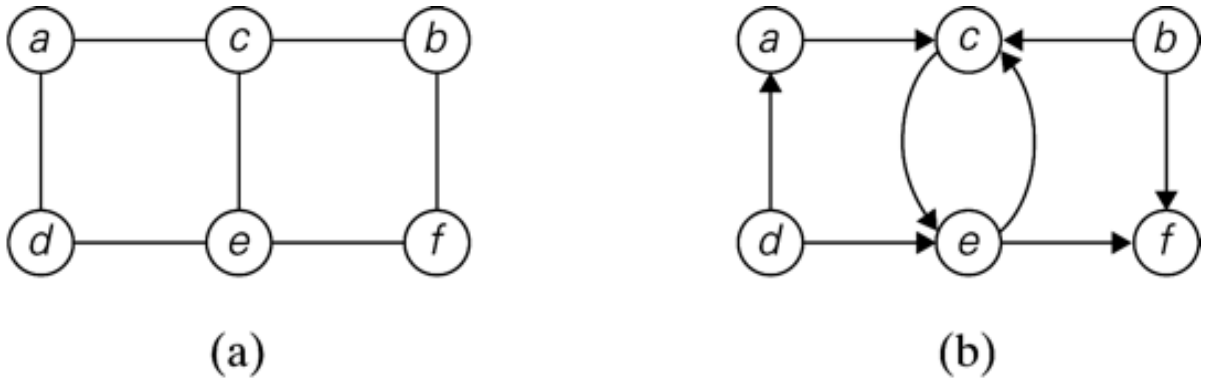
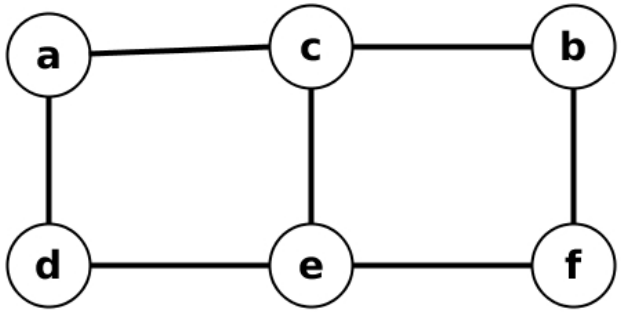


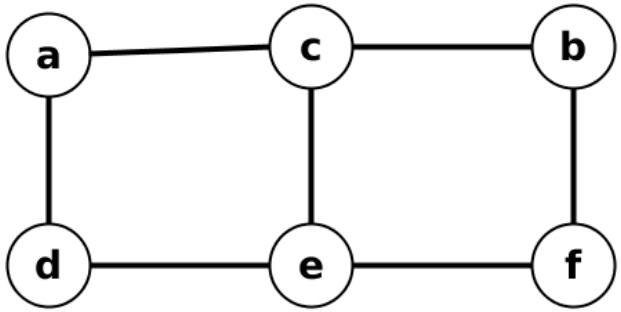

**FIGURE 1.6** (a) Undirected graph. (b) Digraph.

# Graph Representations



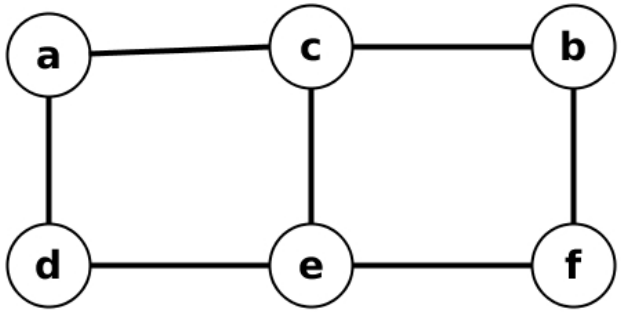

How to represent a graph without drawing it?

# Graph Representations




How to represent a graph without drawing it? Vertex and edge lists (previously)? Other possibilities?

# Graph Representations




How to represent a graph without drawing it? Vertex and edge lists (previously)? Other possibilities?

(a)

| | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| a | 0 | 0 | 1 | 1 | 0 | 0 |
| b | 0 | 0 | 1 | 0 | 0 | 1 |
| c | 1 | 1 | 0 | 0 | 1 | 0 |
| d | 1 | 0 | 0 | 0 | 1 | 0 |
| e | 0 | 0 | 1 | 1 | 0 | 1 |
| f | 0 | 1 | 0 | 0 | 1 | 0 |

(b)

| | | | | | | |
|---|---|---|---|---|---|---|
| a | → | c | → | d | | |
| b | → | c | → | f | | |
| c | → | a | → | b | → | e |
| d | → | a | → | e | | |
| e | → | c | → | d | → | f |
| f | → | b | → | e | | |

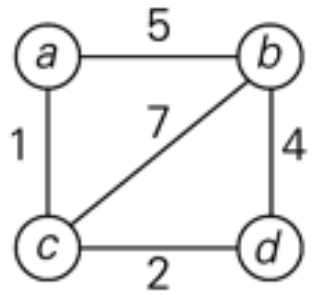
**FIGURE 1.7** (a) Adjacency matrix and (b) adjacency lists of the graph in Figure 1.6a

# Weighted Graphs

What if edges have weights associated with them?

# Weighted Graphs

What if edges have weights associated with them?




(a)

| | a | b | c | d |
|---|---|---|---|---|
| a | ∞ | 5 | 1 | ∞ |
| b | 5 | ∞ | 7 | 4 |
| c | 1 | 7 | ∞ | 2 |
| d | ∞ | 4 | 2 | ∞ |

(b)

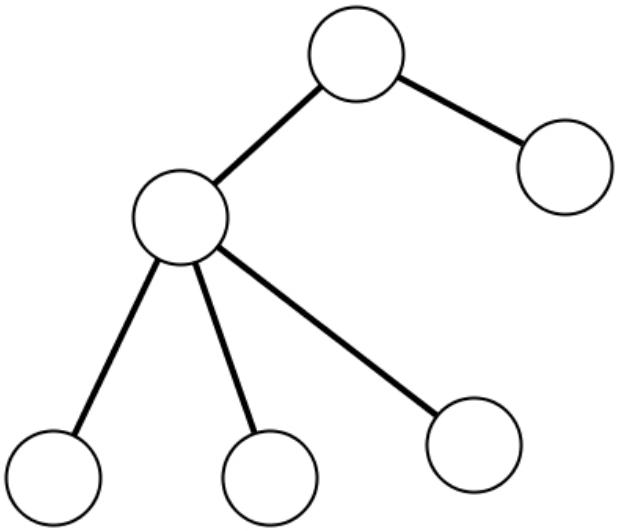
| | |
|---|---|
| a | → b, 5 → c, 1 |
| b | → a, 5 → c, 7 → d, 4 |
| c | → a, 1 → b, 7 → d, 2 |
| d | → b, 4 → c, 2 |

(c)

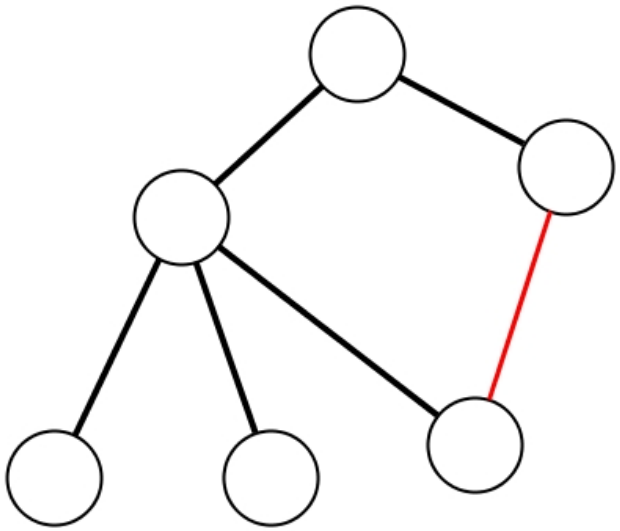
**FIGURE 1.8** (a) Weighted graph. (b) Its weight matrix. (c) Its adjacency lists.

# Trees

A tree is a connected acyclic graph.



(e) tree



(f) Not a tree (but a graph)

# Overview

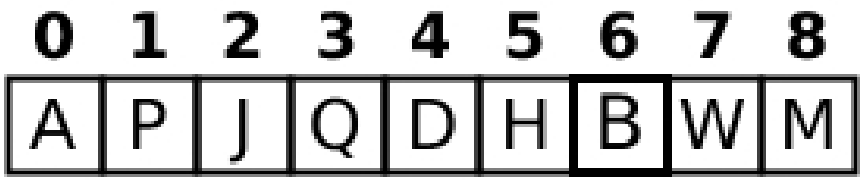

1. Preliminaries
2. What is an algorithm and motivation for its study
3. Abstract Data Types
4. Data Structures
5. Summary

# Data Structures

Now we describe several data structures that can implement different abstract data types.
We will learn more data structures as we progress through the course.

Two important data structures are array and linked-list.
Both can implement many ADT, e.g., sequences, set, dictionary.

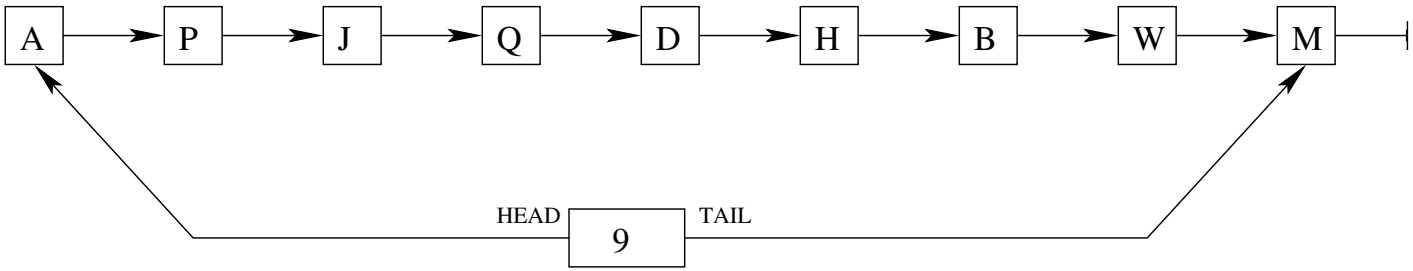
# Array




Characteristics:

- Collection of elements usually stored in a continuous manner, accessed by indices.
- Fast random access.
- Resizing can be costly (may require copying), hence often need to estimate size of array beforehand.

# Linked Lists



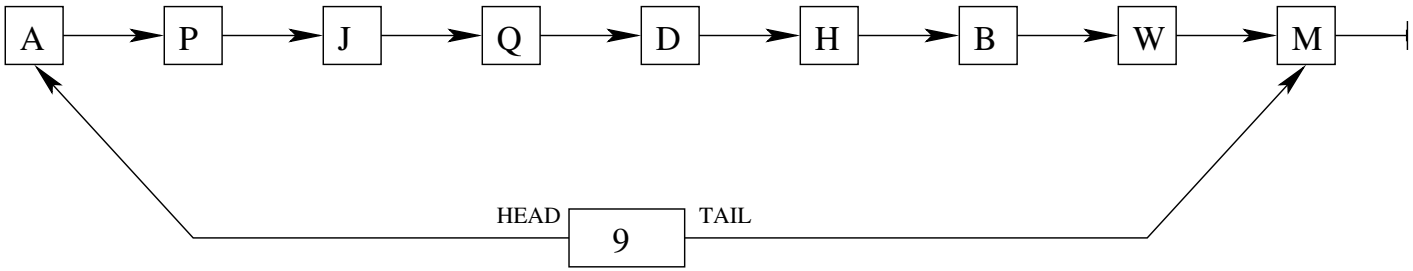

- Each node stores element and a pointer to next node.
- There is a head pointer that points to the start of the list.
- Somtimes there is a tail pointer that points to the end of the list.
- Other variants of linked list (e.g., doubly linked lists, go to labs to see!)

# Linked Lists - insertion

Insert the elements of following sequence of letters, one by one [A,P,J,Q,D,H,B,W,M]:

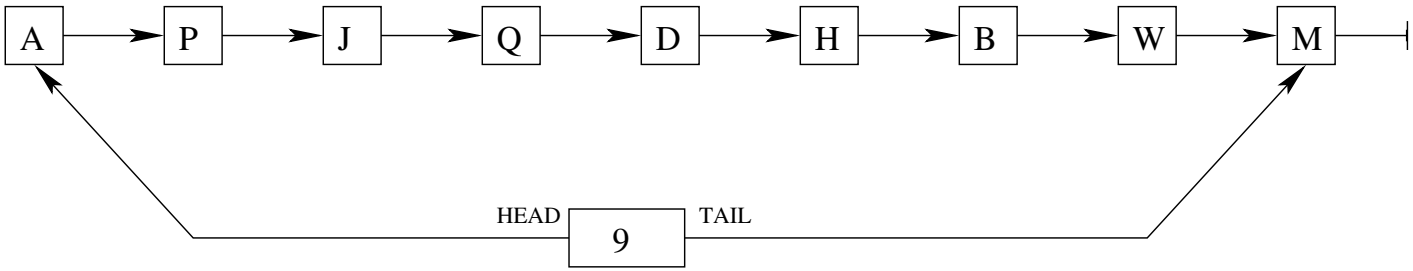
# Linked Lists - insertion

Insert the elements of following sequence of letters, one by one [A,P,J,Q,D,H,B,W,M]:




What if I wanted to insert element C between Q and D?

# Linked Lists - deletion




Given linked-list, delete B:

# Overview


1. Preliminaries
2. What is an algorithm and motivation for its study
3. Abstract Data Types
4. Data Structures
5. Summary

# Summary

- What is an algorithm, examples of it, and why the study of algorithms and data structures are important.
- Discussed the GCD problem and how it can be solved.
- Learnt about abstract data structures and two data structures (arrays and linked lists).

# Summary

- What is an algorithm, examples of it, and why the study of algorithms and data structures are important.
- Discussed the GCD problem and how it can be solved.
- Learnt about abstract data structures and two data structures (arrays and linked lists).
- Ultimately, remember the study of algorithms and data structures is for us to become better at solving problem.

# Todo for next week

What should I be doing?

- Get the textbook.
- Read Chapter 1 and Chapter 2 carefully.
- Watch the video recordings for the Week 2 (Canvas -> Modules -> Week 2).
- Next week (Chapter 2) covers complexity analysis.
- Be ready for next week. Chapter 2 arguably contains some of more difficult material of the course.
- Your workshop this week will focus on the fundamental data structures in Section 1.4.